

GitHub Skills 课程实验报告

GitHub Skills 第 1 节课实验报告

一、基本信息

- 姓名：张君豪
- 学号：2023250361029
- 班级：软件2301B1
- 作业完成日期：_____

二、核心知识点梳理

本节课的重点是认识 Git、GitHub 和仓库的基本概念，并完成课程作业仓库的创建。Git 是一种分布式版本控制工具，主要用于记录文件变化、回退历史版本、比较不同版本之间的差异。GitHub 则是基于 Git 的代码托管与协作平台，它不仅可以保存仓库，还提供 Pull Request、Issues、Actions、Pages 等协作和自动化功能。通过本节学习，我认识到 Git 更偏向本地版本控制工具，而 GitHub 更偏向远程托管平台和协作环境，两者关系密切但不能简单等同。

仓库是 GitHub 中最基础的组织单位。一个仓库可以保存项目代码、实验报告、说明文档和相关资源。创建仓库时需要关注仓库名称、描述、可见性、README、.gitignore 和 License 等选项。仓库名称应当清晰表达用途，描述信息要能让别人快速理解仓库内容。Public 仓库可以被他人访问，适合课程作业展示；Private 仓库则更适合未公开项目或个人练习。

README.md 是仓库的入口文档。别人打开仓库时，首先看到的通常就是 README。它应该说明仓库用途、作者信息、文件目录、使用方式和注意事项。对于课程作业来说，README 不只是简单介绍，还承担着目录索引的作用，方便老师快速找到每一节课的实验报告。

本节还涉及 Markdown 的基本作用。虽然 Markdown 的详细语法放在后续课程继续学习，但在创建 README 时已经能够感受到它的优势：格式简洁、可读性强、适合在 GitHub 网页中直接展示。仓库创建和 README 编写看似是基础操作，但它们决定了整个课程作业的组织方式。

三、学习中的难点

我在学习过程中首先感到困难的是 Git 和 GitHub 的边界。刚开始容易把二者都理解成“上传代码的网站”，但实际使用后发现 Git 可以脱离 GitHub 独立使用，而 GitHub 只是众多远程托管平台中的一种。理解这一点之后，我才明白为什么本地会有 `git status`、`git add`、`git commit` 这些命令，而 GitHub 网页上又会有仓库、分支、Pull Request 等功能。

第二个难点是仓库初始化选项的理解。创建仓库时可以选择添加 README、.gitignore 和 License。README 是说明文件，.gitignore 用来忽略不需要纳入版本控制的临时文件或生成文件，License 用来声明

开源许可。对于本次课程报告仓库，老师要求不添加 .gitignore 和 License，因此只需要初始化 README。这个选择让我认识到仓库配置应当服务于作业要求，而不是看到选项就全部勾选。

第三个难点是 README 内容如何写得规范。README 太短会让仓库显得不完整，太长又会影响查看效率。我认为课程作业仓库的 README 应该突出“信息完整”和“目录清楚”，不需要写成项目宣传页面，也不能只写一行标题。尤其是 6 份实验报告文件名较长，如果没有表格整理，后续检查和提交会比较混乱。

四、实践操作与收获

实践中，我按照课程要求创建了名为 `GitHub_Skills_2023250361029` 的 Public 仓库，并添加 README 文件。仓库描述使用了英文说明，表示这是 GitHub Skills 课程作业仓库。创建完成后，我进一步思考了如何把它整理成一个适合提交的作业仓库，而不是只停留在“建了一个空仓库”的状态。

在整理 README 时，我把个人信息、文件清单、课程主题和使用说明放在仓库首页。这样做的好处是老师或同学进入仓库后，不需要逐个打开文件猜测内容，可以直接通过表格了解每份报告对应的课程主题。这个过程让我意识到文档结构本身也是项目管理的一部分。一个清楚的 README 能降低沟通成本，也能减少作业提交时遗漏文件的风险。

本节实践还让我接触到仓库创建后的基本检查思路。例如创建仓库后要确认可见性是否为 Public，README 是否真的存在，仓库描述是否正确。对于课程作业来说，这些检查虽然简单，但很重要。如果仓库设置为 Private，老师可能无法访问；如果没有 README，仓库首页会缺少说明；如果命名不规范，后续提交和评分都可能受到影响。

建议补充截图位置：可以在此处放置 GitHub 仓库首页截图，展示仓库名称、README 和 Public 状态。

五、学习遇到的困惑与疑问

我比较困惑的问题是：为什么同样是保存文件，GitHub 仓库要比普通网盘更适合作业管理？经过本节学习，我的理解是 GitHub 不只是存储文件，还会记录每次修改的历史，能够展示谁在什么时间修改了什么内容。对于代码和文档类作业，这种可追踪性比普通网盘更有价值。

另一个疑问是：README 应该写到什么程度才算合适？如果内容太详细，可能和具体实验报告重复；如果内容太简单，又不能承担索引作用。我目前的理解是，README 应该重点回答“这个仓库是什么、里面有什么、怎么使用”，具体学习过程和心得则放在各节实验报告中。

我还在思考仓库公开是否会带来信息安全问题。课程作业要求公开仓库时，应避免上传身份证号、手机号、账号密码、访问令牌等敏感信息。学号和姓名属于本次作业要求中的基本信息，可以按要求填写，但其他无关隐私信息不应出现在仓库中。

六、总结

通过第 1 节课，我完成了 GitHub 与 Git 的基础认知，理解了仓库、README、可见性和初始化选项的作用。相比单纯创建一个仓库，我更重要的收获是认识到仓库也是一种组织和表达方式。一个规范的课程作业仓库，应该让查看者能够快速理解内容结构、文件用途和提交状态。本节为后续 Markdown 编写、commit 提交、分支协作和 Issues 管理打下了基础。

GitHub Skills 第 2 节课实验报告

一、基本信息

- 姓名：张君豪
- 学号：2023250361029
- 班级：软件2301B1
- 作业完成日期：_____

二、核心知识点梳理

本节课主要学习 Markdown 基础语法、文件组织方式以及实验报告的规范写法。Markdown 是一种轻量级标记语言，它用简单符号表达标题、列表、表格、代码块、引用和链接等格式。与 Word 这类富文本编辑器相比，Markdown 更关注文本结构，文件本身是纯文本，便于版本控制，也适合在 GitHub、Typora 等工具中查看和导出。

Markdown 标题使用 # 表示层级。一级标题通常用于文档标题，二级标题用于主要章节，三级标题用于小节。标题层级应当连续，不宜从一级标题直接跳到三级标题。列表可以分为有序列表和无序列表，适合整理步骤、要点和检查项。表格适合展示文件清单、任务状态和对比信息。代码块需要使用三个反引号包围，并标注语言，例如：

```
git status
git add .
git commit -m "add experiment report"
```

实验报告不仅要有格式，还要有内容组织。老师要求每份报告包含核心知识点梳理、学习中的难点、实践操作与收获、学习遇到的困惑与疑问四个核心部分。第 4 部分虽然属于选做，但如果认真填写，能够体现学习过程中的真实思考。报告的基本信息也很重要，因为它直接关系到作业识别和归档。

文件组织方面，本次作业需要在仓库根目录下放置 6 份 Markdown 报告、README.md 和 CHECKLIST.md。文件命名采用“课程名 + 第 X 节课 + 姓名 + 学号”的方式，虽然文件名较长，但信息完整，方便老师确认归属。使用统一命名规则也能减少后续整理 PDF 时的混乱。

三、学习中的难点

本节最大的难点是从“会写文字”转变为“会写结构化文档”。以前写实验报告时，更多关注内容本身，格式可以通过 Word 后期调整。但 Markdown 要求在写作时就考虑标题、段落、列表和代码块的关系。比如代码命令如果直接写在段落中，可读性会下降；放入 `bash` 代码块后，既清楚又便于复制。

第二个难点是标题层级的控制。Markdown 中 # 的数量代表标题级别，看起来简单，但如果随意使用，文档结构会变乱。实验报告的一级标题应只有一个，后面的“基本信息、核心知识点梳理”等部分使用二级标题，这样导出 PDF 或在 Typora 中查看大纲时会更清楚。

第三个难点是如何避免内容空泛。Markdown 能帮助排版，但不能自动提高报告质量。如果每份报告都只是写“我学习了 GitHub，感觉很有用”，即使格式正确，也不能体现学习过程。因此在写作时需要具体说明做了什么、遇到什么问题、怎样解决，以及由此得到什么认识。

四、实践操作与收获

实践中，我按照统一结构创建 Markdown 实验报告文件。每份报告都保留基本信息，并把作业完成日期设置为 `_____`，避免自动填写当前日期。这样可以在最终提交前根据老师要求统一补充，防止日期不一致。文件名也严格按照要求保留中文姓名和学号，便于识别。

在写 README.md 时，我使用 Markdown 表格整理 6 份报告的文件名、课程主题和完成状态。表格比普通列表更适合展示清单，因为它能让不同字段对齐，阅读时更直观。CHECKLIST.md 则使用任务列表语法 `- [ ]`，用于提交前逐项检查。这种写法让我认识到 Markdown 不只是写报告，也可以辅助项目管理。

我还练习了在报告中正确书写命令代码块。例如涉及 Git 操作时，使用 `bash` 标注代码块语言，而不是把命令混在正文中。这样做一方面符合 Markdown 格式要求，另一方面也能让读者明确哪些是命令、哪些是说明文字。

在文件组织上，我把所有作业文件放在仓库根目录，避免过深的目录结构。对于只有 6 份报告的课程作业来说，根目录表格索引已经足够清晰。如果以后作业数量增加，可以考虑按课程章节或实验类型建立文件夹，但本次作业不需要过度设计。

建议补充截图位置：可以在此处放置 Typora 打开 Markdown 报告的大纲截图，展示标题层级是否清楚。

五、学习遇到的困惑与疑问

我产生的第一个疑问是：Markdown 是否能够完全替代 Word？目前来看，Markdown 更适合结构清楚、格式相对简单的技术文档和实验报告；如果需要复杂页眉页脚、精细排版或大量图片排版，Word 仍然有优势。因此本课程选择 Markdown，是因为它更适合 GitHub 管理和后续版本控制。

第二个疑问是：代码块是否一定要标注语言？从显示效果看，不标注语言也能显示代码，但标注 `bash`、`markdown` 等语言可以提升可读性，也能让语法高亮更准确。对于课程报告来说，养成标注语言的习惯是更规范的做法。

第三个疑问是：中文文件名在 GitHub 中是否会有问题？实践中 GitHub 可以正常显示中文文件名，但在不同系统和命令行环境中，中文路径有时可能涉及编码问题。因此使用中文文件名时要注意 UTF-8 编码，避免文件名乱码。由于老师已经明确要求按指定格式命名，本次作业应遵守要求。

六、总结

通过第 2 节课，我掌握了 Markdown 的基本写法和实验报告的组织思路。Markdown 的价值不只是让文档好看，更重要的是让内容结构清楚、便于版本控制和在线展示。本节实践让我认识到，写实验报告不能只追求字数，还要关注标题层级、段落组织、代码块规范和文件命名规则。后续在编写 GitHub Skills 课程报告时，我会继续保持统一格式，并尽量让每份报告都有明确主题和真实思考。

GitHub Skills 第 3 节课实验报告

一、基本信息

- 姓名：张君豪
- 学号：2023250361029
- 班级：软件2301B1
- 作业完成日期：_____

二、核心知识点梳理

本节课的重点是 Commit 提交、版本历史和变更追踪。Git 的核心价值之一就是记录项目的变化过程，而 commit 是这个过程中的基本单位。每一次 commit 都可以理解为对仓库当前状态的一次快照，包含修改内容、提交人、提交时间和提交说明。通过 commit 历史，可以了解项目从开始到现在经历了哪些变化。

Git 的常见工作流程包括查看状态、暂存文件、提交版本和查看历史。`git status` 用来查看当前工作区有哪些文件被修改、新增或删除；`git add` 用来把修改放入暂存区；`git commit` 用来把暂存区内容正式保存为一个版本；`git log` 可以查看提交历史。典型命令如下：

```
git status
git add .
git commit -m "add GitHub Skills experiment reports"
git log --oneline
```

工作区、暂存区和版本库是理解 Git 提交机制的关键。工作区是我们实际编辑文件的地方，暂存区是准备提交的内容集合，版本库则保存已经提交的历史记录。Git 不会自动把所有修改都提交，而是要求使用者明确选择哪些内容进入下一次 commit。这种设计可以让提交更有组织性。

提交信息也很重要。好的 commit 信息应该简洁说明本次修改的目的，例如 `add GitHub Skills experiment reports`。不规范的信息如 `update`、`111`、`final` 很难让别人理解修改内容。课程作业虽然规模不大，但仍应养成规范写提交信息的习惯。

三、学习中的难点

本节的第一个难点是理解为什么需要暂存区。刚开始我觉得修改文件后直接提交就可以，为什么还要多一步 `git add`。学习后发现，暂存区可以让我们把多个文件的修改分批组织到不同 commit 中。例如 README 和实验报告可以一起提交，也可以根据任务拆成多个提交。暂存区让版本历史更可控。

第二个难点是判断一次 commit 应该包含多少内容。如果每改一行就提交，历史会过于零散；如果所有修改都堆在一次 commit 中，又不利于回顾和定位问题。比较合理的做法是按任务或功能提交。例如本次作业可以把“创建 6 份实验报告、更新 README、添加 CHECKLIST”作为一个完整任务提交。

第三个难点是理解版本历史的用途。以前我可能只把 Git 当作上传工具，认为提交只是为了把文件放到 GitHub 上。但实际上 commit 历史可以帮助回退错误、比较版本、审查修改，也能体现作业完成过程。特别是在团队协作中，清楚的历史记录比单个最终文件更有价值。

四、实践操作与收获

在实践中，我先查看仓库状态，确认当前有哪些文件需要创建或修改。随后创建 6 份实验报告、更新 README，并添加 CHECKLIST。完成编辑后，再通过 Git 状态检查确认这些文件都被 Git 识别。这个过程让我意识到，提交前查看状态是一种必要习惯，不能凭感觉认为所有文件都已经准备好。

我还练习了使用统一的提交信息。本次要求 commit 信息为 `add GitHub Skills experiment reports`，它能够清楚说明本次提交的主要内容。相比随意写“作业”或“完成”，这个信息更适合出现在版本历史中。以后查看提交记录时，可以快速知道这次提交是添加 GitHub Skills 实验报告。

在理解变更追踪时，我认识到 Git 不只是保存最终文件，还能保存修改过程。如果后续发现某份报告内容有误，可以通过历史记录查看当时是如何修改的；如果 README 表格不小心被改坏，也可以从历史版本中恢复。这样的能力对于长期维护文档和代码都很重要。

实践中也要注意不要把无关文件提交到仓库。例如本地可能存在临时文件、导出的 PDF、编辑器缓存等。如果不加检查就执行 `git add .`，可能会把无关内容一起提交。虽然本次作业要求 `.gitignore` 为 `None`，但这不代表可以随意提交所有本地文件。提交前仍应检查文件清单。

建议补充截图位置：可以在此处放置 `git status` 或 GitHub 提交历史页面截图，展示提交前后的状态变化。

五、学习遇到的困惑与疑问

我比较困惑的是：如果已经提交了错误内容，应该如何处理？目前了解到有多种方法，例如继续提交一个修复 commit，或者在特定情况下使用回退命令。对于课程作业来说，如果错误已经推送到远程仓库，通常更稳妥的方式是新增一次修复提交，而不是强行改写历史。

另一个疑问是：commit 信息应该用中文还是英文？从课程作业角度看，中文可以表达清楚；从开源社区和工具习惯看，英文短语更常见。本次老师指定了英文提交信息，因此应按要求执行。我认为最重要的是信息清楚、准确、可理解，而不是机械追求某一种语言。

我还在思考 `git add .` 是否总是合适。它很方便，但也可能把所有未跟踪文件都加入暂存区。如果仓库中存在不应提交的文件，使用 `git add .` 就有风险。更规范的做法是在提交前先运行 `git status`，必要时使用 `git add README.md` 或指定文件名进行暂存。

六、总结

通过第 3 节课，我理解了 commit 在 Git 中的核心地位，也认识到版本历史不是形式上的记录，而是项目管理和协作的重要依据。一个好的提交应该内容集中、信息清楚、范围明确。本节实践让我从“把文件上传到 GitHub”转变为“用 Git 管理修改过程”。后续进行分支和 Pull Request 学习时，commit 历史也会成为协作审查的重要基础。

GitHub Skills 第 4 节课实验报告

一、基本信息

- 姓名：张君豪
- 学号：2023250361029
- 班级：软件2301B1
- 作业完成日期：_____

二、核心知识点梳理

本节课主要学习 Branch 分支管理和 Pull Request 协作流程。分支是 Git 中非常重要的概念，它允许我们在不直接影响主分支的情况下进行修改。主分支通常保存相对稳定的内容，而新功能、实验报告修改或问题修复可以放在单独分支中完成。完成并检查后，再通过合并把修改带回主分支。

创建分支的常见命令如下：

```
git checkout -b report/github-skills-homework
git status
git push -u origin report/github-skills-homework
```

Pull Request 简称 PR，是 GitHub 上进行代码审查和合并的重要机制。它不是简单的“提交按钮”，而是一个讨论、检查和合并修改的过程。创建 PR 时通常需要填写标题和描述，说明本次修改包含哪些内容、为什么修改、是否还有未完成事项。其他人可以在 PR 中查看文件差异、评论、提出修改建议，最后再决定是否合并。

分支和 PR 的配合能够降低直接修改主分支带来的风险。对于课程作业来说，虽然只有个人完成，也可以通过新分支和 PR 模拟真实协作流程。这样做能够展示自己理解了 GitHub 的标准工作方式，而不是只会在网页上直接编辑文件。

三、学习中的难点

本节的第一个难点是理解分支的意义。刚开始可能会觉得只有一个人做作业，没有必要创建分支。但学习后我意识到，分支不只是多人协作才需要，个人开发中也很有用。它可以把正在修改的内容和稳定版本隔离开，避免未完成的修改直接影响主分支。

第二个难点是 Pull Request 与直接提交的区别。直接提交到 main 分支速度快，但缺少检查和讨论过程。PR 则提供了一个缓冲区，可以在合并前查看所有变更。尤其是当修改文件较多时，PR 的文件差异页面可以帮助发现遗漏、格式问题和不必要的修改。

第三个难点是分支命名。分支名需要表达任务含义，例如 `report/github-skills-homework` 表示这是课程报告作业分支。相比 `test`、`new` 这类名称，规范分支名能让别人快速理解分支用途，也便于后续管理。如果项目分支很多，命名混乱会明显增加沟通成本。

四、实践操作与收获

本次实践中，我优先创建新分支 `report/github-skills-homework`，并在该分支上完成实验报告相关文件的创建与整理。这样主分支保持原状，作业内容集中在独立分支中。完成后再提交 commit，并通过 Pull Request 向 main 分支提出合并请求。

在准备 PR 描述时，我把本次修改内容概括为：创建 6 份实验报告、创建 README.md、创建 CHECKLIST.md、作业完成日期暂未填写、后续可用 Typora 导出 PDF。这种描述方式比只写“完成作业”更清楚，也方便老师或自己回顾本次 PR 的范围。

通过这个过程，我认识到 PR 的价值在于“让修改可审查”。如果直接把所有文件放进 main 分支，虽然也能完成作业，但缺少一个集中查看差异的入口。PR 页面可以展示新增文件、修改内容、提交记录和讨论记录，符合现代软件开发中的协作习惯。

我还注意到，在个人仓库中使用 PR 也能帮助自我检查。例如提交后进入 PR 的 Files changed 页面，可以逐个查看 Markdown 文件是否缺少标题、是否误填日期、是否存在格式问题。即使没有其他同学参与审查，这个过程也能提高提交质量。

建议补充截图位置：可以在此处放置 Pull Request 页面截图，展示分支名、PR 标题和文件差异。

五、学习遇到的困惑与疑问

我比较疑惑的是：什么时候应该创建分支，什么时候可以直接在 main 上修改？目前我的理解是，如果只是修正一个很小的错别字，并且项目规则允许，直接修改可能可以接受；但如果涉及多个文件、作业提交、新功能或可能影响主线内容的修改，使用分支和 PR 更稳妥。

另一个问题是：PR 合并后分支是否需要删除？在实际项目中，已经合并的功能分支通常可以删除，避免分支列表越来越乱。但如果课程需要保留分支作为作业过程证据，也可以暂时保留。是否删除应根据课程要求或项目规范决定。

我还在思考 PR 审查主要看什么。除了代码是否能运行，文档类作业也需要审查标题层级、文件命名、内容完整性、是否有无关文件、是否误填日期等。也就是说，PR 审查并不只适用于程序代码，同样适用于报告和说明文档。

六、总结

通过第 4 节课，我理解了分支和 Pull Request 在 GitHub 协作中的作用。分支让修改过程与主分支隔离，PR 让修改过程可检查、可讨论、可追踪。即使本次作业由个人完成，使用分支和 PR 仍然有助于模拟真实开发流程，也能让作业提交更规范。本节学习让我认识到，GitHub 的价值不仅在于保存文件，还在于提供了一整套协作流程。

GitHub Skills 第 5 节课实验报告

一、基本信息

- 姓名：张君豪
- 学号：2023250361029
- 班级：软件2301B1
- 作业完成日期：_____

二、核心知识点梳理

本节课主要学习 Issues、项目任务管理与协作沟通。Issues 是 GitHub 中用于记录问题、任务、需求和讨论的重要功能。它既可以用于软件项目中的 bug 反馈，也可以用于课程作业中的任务拆分和进度记录。一个 Issue 通常包括标题、正文、标签、负责人、评论和状态等信息。

Issues 的价值在于把零散的问题变成可追踪的任务。例如“补充第 3 节课报告内容”“检查 README 表格”“确认作业完成日期是否填写”等事项，都可以用 Issue 记录。相比把任务记在聊天记录或纸上，Issue 更容易和仓库文件、提交记录、Pull Request 关联起来。

项目管理中常用的概念包括任务拆分、优先级、状态跟踪和沟通记录。GitHub Issues 可以配合 labels 标记类型，例如 documentation、bug、question 等；也可以使用 assignees 指定负责人；还可以通过评论记录讨论过程。对于多人协作项目，这些功能能减少口头沟通遗漏。

本节还涉及沟通表达的规范。一个好的 Issue 标题应当简洁明确，正文应说明背景、期望结果和可能的处理方式。例如标题写“补充 CHECKLIST 提交前检查项”比写“问题”更有意义。沟通清楚本身也是软件工程能力的一部分。

三、学习中的难点

本节的第一个难点是理解 Issue 和 Pull Request 的区别。Issue 更偏向记录问题或任务，PR 更偏向提交具体修改。一个 Issue 可以先提出“需要补充实验报告”，然后通过一个 PR 来解决它。PR 合并后，可以关闭对应 Issue。二者配合使用，能让问题提出、修改实现和结果确认形成闭环。

第二个难点是避免 Issue 过于随意。如果标题和描述都很模糊，Issue 就失去了管理价值。例如只写“改一下”很难让别人知道要改什么。比较好的写法是说明具体文件、具体问题和期望结果。即使是个人作业，也应该练习这种清楚表达。

第三个难点是任务拆分的粒度。任务太大时，Issue 难以推进；任务太小时，Issue 数量又会过多。对于本次 6 份实验报告，可以按“创建报告文件”“完善 README”“提交前检查”这样的粒度拆分，而不必为每一段文字创建一个任务。

四、实践操作与收获

在本次课程作业整理过程中，我把 CHECKLIST.md 作为一种轻量任务管理文件。虽然它不是 GitHub 网页中的 Issue，但它体现了类似的检查思想：把提交前容易遗漏的事项列成清单，并逐项确认。例如检查 6 份报告是否完成、基本信息是否一致、作业完成日期是否保持空白、Markdown 标题层级是否正确、文件命名是否符合要求。

如果使用 GitHub Issues，可以进一步把这些检查项拆分成任务。例如创建一个 Issue，标题为“检查 6 份 GitHub Skills 实验报告格式”，正文列出需要确认的项目。完成检查后，在 PR 中引用该 Issue，形成从任务提出到修改完成的记录。这个流程让我理解到，Issues 并不只是报告 bug，它也可以用于课程作业和文档管理。

我还认识到，沟通记录对团队协作非常重要。很多时候问题不是技术不会，而是没有说明清楚。例如“第 4 节课需要写 PR 流程”这个要求，如果只记在脑子里，很容易遗漏；写成 Issue 或 checklist 后，任务就变得具体可见。对于课程小组作业，Issues 可以帮助成员分工，也方便老师了解每个人参与情况。

在实践收获方面，我认为本节让我从“完成文件”进一步转向“管理任务”。当作业文件数量增加时，光靠记忆很容易出错。使用 Issues、README 表格和 CHECKLIST，可以让作业状态更加清楚，也能减少提交前的紧张和返工。

建议补充截图位置：可以在此处放置 GitHub Issues 页面或 CHECKLIST.md 页面截图，展示任务管理方式。

五、学习遇到的困惑与疑问

我比较困惑的是：个人作业是否真的需要 Issues？从效率上看，简单作业可能不需要创建很多 Issue；但从学习 GitHub 协作流程的角度看，至少了解 Issue 的作用是必要的。它可以帮助我们把问题管理起来，而不是只依赖聊天或口头说明。

另一个疑问是：Issue 是否必须和代码修改绑定？学习后我理解到不一定。Issue 可以是问题、建议、讨论或计划，不一定马上对应代码修改。但如果 Issue 最终通过某个 PR 解决，把它们关联起来会更清楚。

我还在思考如何写出高质量的 Issue。一个有价值的 Issue 应该包含背景、现象、预期结果和补充信息。对于文档任务，也可以包含涉及文件、检查标准和完成条件。这样别人看到后才能快速判断如何处理，而不需要反复追问。

六、总结

通过第 5 节课，我理解了 GitHub Issues 在任务管理和协作沟通中的作用。Issues 可以把问题、需求和待办事项变成可追踪记录，配合 PR 能形成完整的工作闭环。本节实践让我认识到，软件工程不仅是写代码或写报告，还包括任务拆分、状态检查和清晰沟通。对于本次课程作业，README、CHECKLIST 和 PR 描述都体现了这种管理思想。

GitHub Skills 第 6 节课实验报告

一、基本信息

- 姓名：张君豪
- 学号：2023250361029
- 班级：软件2301B1
- 作业完成日期：_____

二、核心知识点梳理

本节课主要了解 GitHub Pages 或 GitHub Actions 的初步实践，并对 GitHub Skills 学习进行总结。GitHub Pages 是 GitHub 提供的静态网页托管功能，可以把仓库中的 Markdown、HTML、CSS 等内容发布为网页。它常用于个人主页、项目文档、课程展示和简单网站。对于课程实验报告来说，Pages 可以作为扩展展示方式，但本次主要提交形式仍然是 Markdown 文件。

GitHub Actions 是 GitHub 提供的自动化工作流工具。它可以在特定事件发生时自动执行任务，例如 push 后运行测试、PR 创建后检查格式、发布版本时构建项目等。Actions 的配置通常写在 `.github/workflows` 目录下的 YAML 文件中。一个简单的工作流会包含触发条件、运行环境和执行步骤。

Pages 和 Actions 都体现了 GitHub 从代码托管平台向开发协作平台的扩展。前面几节课学习了仓库、Markdown、commit、branch、PR 和 Issues，本节则进一步认识到 GitHub 还可以承担自动化和发布功能。对于软件工程学习来说，这些功能能够帮助项目更规范、更高效。

需要注意的是，本次实验报告不能伪造真实运行结果。如果没有实际配置 Pages 或 Actions，就不能写成“已经成功部署网页”或“自动化测试全部通过”。可以写了解了基本概念、适合的应用场景、后续可以如何补充实践，并在需要时预留截图位置。

三、学习中的难点

本节的第一个难点是理解 GitHub Actions 的触发机制。Actions 不是手动运行脚本那么简单，它通常由事件触发，例如 `push`、`pull_request` 或手动触发。不同触发条件适合不同场景：`push` 后检查提交，PR 时进行审查，定时任务则适合周期性操作。理解触发条件是使用 Actions 的第一步。

第二个难点是 YAML 配置格式。YAML 对缩进敏感，层级错误可能导致工作流无法识别。与 Markdown 相比，YAML 更像配置文件，要求字段准确、缩进一致。对于初学者来说，即使逻辑很简单，也可能因为少一个空格或写错字段而失败。

第三个难点是区分 Pages 和 Actions 的用途。Pages 面向展示，Actions 面向自动化。Pages 可以把内容发布成网页，Actions 可以检查、构建或部署内容。二者可以结合使用，例如通过 Actions 构建静态网站并发布到 Pages，但它们不是同一个功能。

四、实践操作与收获

在本次课程作业中，我没有编造 Pages 或 Actions 的实际运行结果，而是把它们作为第 6 节课的学习主题进行整理。根据作业要求，本仓库重点是 6 份 Markdown 实验报告、README 和 CHECKLIST。后续如果老师要求展示网页，可以进一步开启 GitHub Pages，把仓库文档发布为静态页面。

如果后续补充 Actions，可以考虑添加一个简单的 Markdown 检查工作流，例如在 PR 创建时检查 Markdown 文件是否存在、文件名是否符合要求、是否包含基本信息和必要标题。这个场景比复杂自动化更贴近课程作业，也能体现 Actions 的实际价值。

本节实践让我认识到，工具学习需要区分“了解概念”和“真实执行”。报告中可以写自己理解了 Pages 和 Actions 的用途，也可以写计划如何使用，但不能把未发生的操作写成已经完成。保持真实记录是实验报告的基本要求，也能避免后续检查时出现无法解释的问题。

通过 6 节课的整体学习，我把 GitHub 的基础功能串联起来：先创建仓库和 README，再用 Markdown 组织报告，通过 commit 记录修改，用 branch 和 PR 管理合并过程，用 Issues 或 CHECKLIST 管理任务，最后了解 Pages 和 Actions 的扩展能力。这条学习路径比较完整，能够覆盖课程作业所需的主要知识点。

建议补充截图位置：如果后续实际启用 GitHub Pages 或 Actions，可以在此处补充 Pages 设置页面或 Actions 运行页面截图。

五、学习遇到的困惑与疑问

我比较疑惑的是：什么时候值得使用 GitHub Actions？如果只是一个很小的文档仓库，复杂 Actions 可能没有必要；但如果作业需要反复检查格式、自动构建 PDF 或部署网页，Actions 就能节省人工检查时间。关键是自动化应当解决真实问题，而不是为了使用工具而使用工具。

另一个问题是：GitHub Pages 是否适合提交课程报告？如果老师要求 PDF 或 Markdown 文件，Pages 只能作为辅助展示，不能替代指定提交格式。但 Pages 可以让报告更方便在线阅读，也可以作为个人学习成果展示。是否启用应以课程要求为准。

我还在思考自动化检查的边界。Actions 可以检查文件是否存在、命名是否正确、格式是否大致符合要求，但它很难判断报告内容是否真实、是否有深度。这说明自动化工具可以提高效率，但不能完全替代人的认真学习和审查。

六、总结

通过第 6 节课，我初步认识了 GitHub Pages 和 GitHub Actions 的作用，并对整个 GitHub Skills 学习过程进行了梳理。GitHub 不只是保存文件的平台，还提供文档展示、协作管理和自动化能力。本次 6 份实验报告让我从基础概念逐步过渡到完整工作流：仓库、Markdown、commit、branch、PR、Issues、Pages 和 Actions。后续如果继续深入学习，我希望能实际配置一次 Actions 工作流，用自动化方式检查课程报告格式。